

## Experiment # 11

### Image Transformation into Frequency Domain

#### Objective

1. To transform into frequency domain

#### Software

- Python Idle 3.7 or 3.8.5
- Pycharm

#### Theory

**Fourier Series** tells us that any function can be represented as a sum of sines/cosines of different frequencies multiplied by a different coefficient. Similarly, non-periodic functions can also be represented as the integral of sines/cosines multiplied by weighing function.

The Fourier transform of  $f(x)$  is given by

$$\mathcal{F}\{f(x)\} = F(u) = \int_{-\infty}^{\infty} f(x) \exp[-j2\pi ux] dx$$

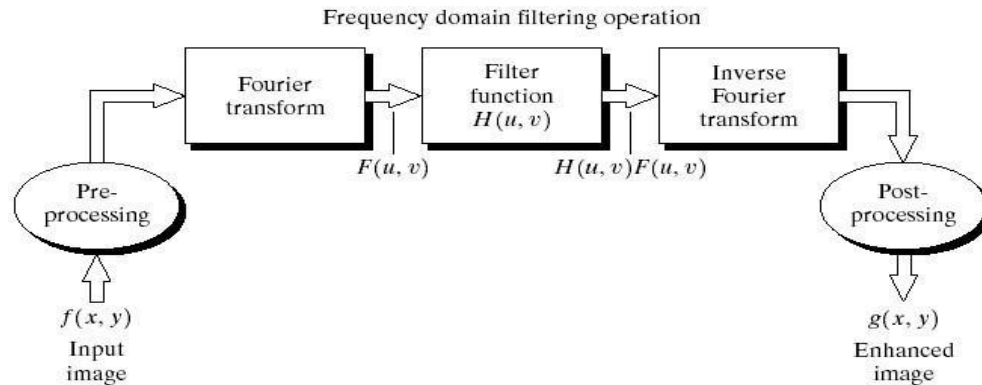
Fourier transform pair for a function  $f(x,y)$  of two variables;

$$\mathcal{F}\{f(x,y)\} = F(u,v) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x,y) \exp[-j2\pi(ux + vy)] dx dy$$

$$\mathcal{F}^{-1}\{F(u,v)\} = f(x,y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} F(u,v) \exp[j2\pi(ux + vy)] du dv$$

The filtering in frequency domain consists of following steps:

1. Compute  $F(u,v)$  the DFT of the image
2. Multiply  $F(u,v)$  by a filter function  $H(u,v)$  Compute the inverse DFT of the result



### Some Useful Commands:

1. To obtain the Fourier Transform of an image: `my_transformed_image = numpy.fft.fft2(my_image)`
2. To obtain the Inverse Fourier Transform of an image: `my_inverse_image = numpy.fft.ifft2(my_image)`
3. To shift the DC component of a Fourier Transformed Image to center: `my_shifted_image = numpy.fft.fftshift(my_transformed_image)`
4. To shift the DC component back to the top left corner: `my_inverse_shifted_image = np.fft.ifftshift(my_shifted_image)`
5. To calculate absolute of a value: `my_absolute = numpy.abs(my_image)`
6. To calculate the exponential of an element: `my_exponential = numpy.exp(my_input)`
7. To use the value of pi: `numpy.pi`
8. To denote a complex number: **simply put j after it e.g. -1j**
9. To multiply two matrices point by point: `my_result = numpy.multiply(my_image, my_filter)`
10. To normalize an image: `my_image_normalized = cv2.normalize(my_source_image, my_destination_image, new_min_value, new_max_value, NORM_MINMAX, CV_8UC1)`

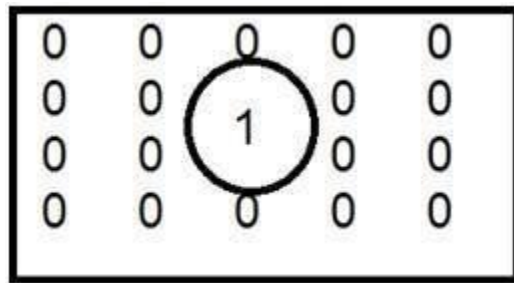
The argument `NORM_MINMAX` tells the function which method to use to normalize the image while the argument `CV_8UC1` tells the number of channels that the destination image will have. In other words, `CV_8UC1` means a grayscale image in unit8.

## High pass frequency components and Low pass frequency components

The high pass frequency components denotes edges whereas the low pass frequency components denotes smooth regions.

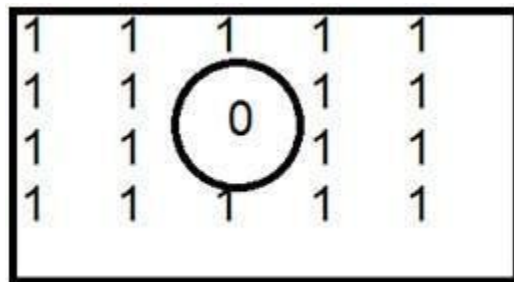
### Ideal low pass and Ideal High pass filters

This is the common example of low pass filter.

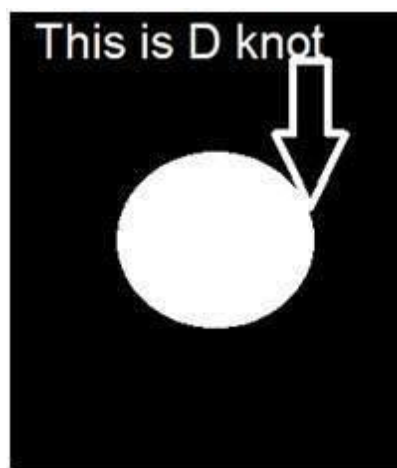


When one is placed inside and the zero is placed outside, we get a blurred image. Now as we increase the size of 1, blurring would be increased and the edge content would be reduced.

This is a common example of high pass filter.



When 0 is placed inside, we get edges, which gives us a sketched image. An ideal low pass filter in frequency domain is given below.



**Lab Task:**

1. Apply High pass filter on the frequency spectrum of an image.
2. Create an ideal low pass and high pass filter. User should decide which should be applied. Take DFT of the image and multiply the transformed image and the filter. Take inverse DFT of the image and display the results